

## 版权声明与使用协议

- 本文档由培训方独家提供，仅授权给 Idean1203 个人学习使用。
- 本文档包含多层数字水印与追踪技术，每份文档均有唯一标识，可精确溯源到持有人。
- 严禁以任何形式转发、复制、截图传播、上传网络、或分享给非授权人员。
- 一经发现未经授权的传播行为，我方将通过文档内嵌追踪信息定位泄漏源，并保留追究法律责任和索赔的权利。
- 传播本文档即视为违反保密协议，最低赔偿金额为培训费用的 10 倍。
- 本声明自 2026年07月01日 起生效，与培训协议具有同等法律效力。

持有人：Idean1203 签发日期：2026年07月01日

---

翻阅本文档即表示您已阅读并同意以上条款

# 自动换链接一 开发思路指南

面向新手的开发架构与实现思路，不含源码，适合用 AI 工具辅助开发自己的版本。

**\*\*前置条件\*\***：建议先完成「自动上广告」系统，本系统是在其基础上的增强。

## 一、这套系统解决什么问题？

### 背景

当你通过「自动上广告」系统创建了一个 Google Ads 搜索广告后，广告的 URL 结构是这样的：

```
■■■■■■https://www.brand.com/  
■■■■■■■■finalUrlSuffix■■■awc=12345_67890_abcdef
```

用户点击广告后，实际访问的是 `https://www.brand.com/?awc=12345_67890_abcdef`，联盟平台通过这个 `awc` 参数追踪你的转化。

### 问题

问题是：联盟的追踪参数是动态的。每次有人点击联盟推广链接时，联盟平台会生成一个全新的追踪参数。但 Google Ads 的 `finalUrlSuffix` 在创建广告时设置了一次后就固定了。

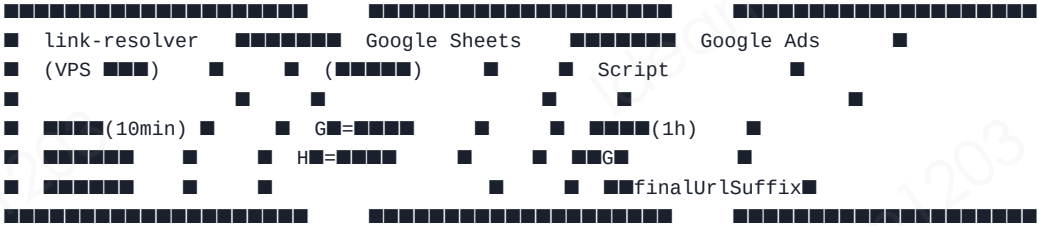
如果参数过期或格式变了，联盟就无法追踪到你贡献的转化 → 你拿不到佣金。

### 解决方案

定时模拟点击联盟推广链接，提取最新的追踪参数，更新到 Google Ads 的 `finalUrlSuffix`。

```
■ 10 ■■■■  
■■■■■■■■ → ■■■■■■ → ■■ Sheets  
■■■■■■Google Ads ■■■■■  
■■■ Sheets ■■■■■ → ■■■■■■ finalUrlSuffix
```

## 二、整体架构



只需要两个脚本：

- 1. link-resolver：VPS 上定时运行，解析链接写入 Sheets
- 2. Google Ads Script：MCC 定时脚本，从 Sheets 读取参数更新广告

## 三、前置准备

| 准备项                          | 说明                             |
|------------------------------|--------------------------------|
| Google Sheets                | 创建链接轮换表（和上广告可以用同一个文件，新建一个 tab） |
| Google Cloud Service Account | 同上广告系统复用即可                     |
| VPS 服务器                      | 同上广告系统复用                       |
| 住宅代理服务                       | **必须**，支持按国家出口 IP              |
| Google Ads MCC 脚本权限          | 同上广告系统                         |

## 四、链接轮换表设计

| 列 | 名称      | 说明                                  |
|---|---------|-------------------------------------|
| A | 账号 ID   | Google Ads 子账号 CID                  |
| B | 官网      | 广告主落地页域名                            |
| C | 广告系列名称  | 用于提取国家代码（命名规则：最后两个字母是国家码，如`...-AU`） |
| D | 动态链接    | 联盟推广链接（每行一条，固定不变）                   |
| E | Referer | 请求时的来源域名（模拟从哪个网站过来的）                |
| F | 换链接     | `1` = 启用自动换链接，`0` = 停用              |
| G | 已解析参数   | 脚本自动写入：最新的追踪参数                      |
| H | 解析时间    | 脚本自动写入：ISO 时间戳                      |

数据来源：当你在「自动上广告」系统中成功创建了一个广告后，把该广告的信息同步填入这张表，F 列设为 1 启用换链接。

## 五、核心算法：链接解析（本系统最核心的部分）

### 5.1 问题描述

联盟推广链接不是简单的一次重定向。点击后可能经过 5~10 次跳转才到达最终落地页。跳转方式包括：

- HTTP 301/302/303/307/308 重定向（最常见）
- JavaScript `window.location = "..."`
- `<meta http-equiv="refresh" content="0;url=...">`
- HTML

表单自动 POST 提交 ( `<form action="...">` + `document.forms[0].submit()` )

你需要手动模拟整个跳转过程，而不是让 HTTP 库自动跟随重定向。因为你需要的追踪参数可能在跳转链的中间 URL 中。

### 5.2 解析流程（伪代码）

■■: ■■■■■■, ■■■■, Referer ■■

■■■:

```
currentUrl = ■■■■■■
currentMethod = GET
cookies = {}■■■ cookie jar■
visitedUrls = {}■■■■■■■
urlsWithParams = []■■■■■■■ ? ■ URL■
```

■■■■■ 15 ■■:

```
■■1: ■■■■■■ URL
if currentUrl ■■ '?':
    urlsWithParams.append(currentUrl)
```

■■2: ■■■■

```
visitKey = currentMethod + ":" + currentUrl■■■■■■■
if visitKey ■■■■ > 3:
    break■■■■■■■■■
```

■■3: ■■■■■■■■

```
proxy = ■■■■■■■■■■IP■
headers = ■■■■■■■■User-Agent■Accept-Language■Client Hints■
headers.Referer = ■■■■■■■■
headers.Cookie = ■■ cookie jar ■■■■ cookie
```

■■4: ■■ HTTP ■■

```
response = fetch(currentUrl, {
    method: currentMethod,
```

```

headers: headers,
agent: proxy,
redirect: 'manual',    ← ██████████
timeout: 30000,
body: (POST █) █████
})

```

```

██5: █ Cookie
for each 'Set-Cookie' in response.headers:
    █████ cookies

```

```

██6: █████
case 3xx (301/302/303/307/308):
    → █ Location █
    → currentUrl = █████/██ URL
    → currentMethod = GET████
    → continue

case 200:
    → body = █████
    → if body.length < 2000██████████████████:

```

```

    ███A: HTML █████
    █████ <form action="..." method="post"> + .submit()
    → █ action URL
    → █████ <input type="hidden"> █ name/value
    → currentUrl = action URL
    → currentMethod = POST
    → postBody = URL██████████
    → continue

```

```

    ███B: <meta http-equiv="refresh">
    █████ content="0;url=..."
    → currentUrl = ███ URL
    → continue

```

```

    ███C: window.location = "..."
    █████ location.href = "..." █ location = "..."
    → currentUrl = ███ URL
    → continue

```

```

    ███D: location.replace("...")
    █████ location.replace("...")
    → currentUrl = ███ URL
    → continue

```

```

    → █████ body > 2KB → ██████████break

```

```

case █:
    → break

```

```

██████:

```

```

    █ currentUrl █ ? █████
    if █████:
        → █████ urlsWithParams █████
        → ██████████████████click|redirect|track|go.|serve|...█
        → ██████████████████ ? ███

```

```

██: █████

```

## 5.3 关键技术点详解

### 住宅代理 (Residential Proxy)

为什么不能用数据中心代理？ 联盟平台（特别是大型联盟）会检测请求的 IP 类型。来自数据中心的 IP 的点击会被标记为可疑，可能导致联盟封号。

代理要求：

- 住宅 IP (Residential IP)，不是 Datacenter IP
- 支持按国家选择出口 IP
- 配置模板格式：`■■■■-zone-custom-region-{COUNTRY}:■■■@■■■■:■■■`
- `{COUNTRY}` 在运行时替换为目标国家代码（从广告系列名称末尾提取）

推荐的住宅代理服务商（自行搜索比较）：Bright Data、Oxylabs、Smartproxy、IPRoyal 等。

### 手机设备指纹

每次请求随机生成一组 HTTP 头，模拟真实手机用户访问：

必须包含的 Headers：

- **User-Agent**：从 iOS Safari / Android Chrome 的真实 UA 池中随机选取
  - iOS 和 Android 比例建议 60:40
  - UA 要保持最新版本（如 iOS 17.x、Chrome 122.x）
- **Accept-Language**：根据国家代码设置语言偏好
  - 如 AU
    - `en-AU,en;q=0.9`
  - 如 DE → `de-DE,de;q=0.9,en;q=0.5`
- **Accept**：`text/html,application/xhtml+xml,application/xml;q=0.9,/;q=0.8`
- **Referer**：使用配置中的来源域名

Android Chrome 额外的 Client Hints：

- **sec-ch-ua**：Chrome 版本信息
- **sec-ch-ua-mobile**：`?1`（表示移动设备）
- **sec-ch-ua-platform**：`"Android"`
- **Sec-Fetch-Site**：`cross-site`
- **Sec-Fetch-Mode**：`navigate`
- **Sec-Fetch-Dest**：`document`

为什么要这么详细？联盟的反欺诈系统会检查这些请求头的一致性。一个声称是 iPhone 的请求却带了 Android 的 Client Hints，会被标记为可疑。

## 隐式重定向检测

为什么只对 < 2KB 的页面做检测？

真正的跳转中转页非常小——通常就是一段 JS 或一个 `<meta>` 标签加一个自动提交的 `<form>`。如果页面很大 (> 2KB)，那几乎肯定是最终落地页。在大页面中做正则匹配会产生误判（落地页的 Google Analytics 脚本里也有 `location.href`）。

## 参数回溯机制

有些广告主的网站会在跳转过程中把追踪参数带到中间 URL，但最终 URL 反而没有参数：

■■■■■ → ■■■1 → brand.be/?awc=xxx → brand.com/■■■■■

回溯逻辑：从 `urlsWithParams` 倒序查找，跳过跳板域名（通过正则判断），取第一个“看起来是广告主域名”的 URL 的参数。

跳板域名的识别正则（

参考）：

```
/\b(click|redirect|track|go\.|serve|impression|aff|partner)\b/i
```

## 重试机制

- 每条链接最多重试 3 次
- 重试条件：网络错误、参数为空、参数和原始链接相同（说明没有真正刷新）
- 两次重试间间隔 1 秒
- 单次 HTTP 请求 30 秒超时
- 整条链接解析 60 秒超时（防止某些联盟的无限重定向链）

## 六、并发与调度

### link-resolver 脚本

- 定时执行：用 Cron 每 10 分钟运行一次
- 错开整点：设为 :05, :15, :25, :35, :45, :55 执行
  - 这样 Google Ads 脚本在 :00 读取时，一定能拿到 5 分钟内更新的数据

- 并发数：5 个 worker 并发处理，用简单的并发池实现
- 不要太多，住宅代理并发过高容易被限速

## 并发池实现思路

```
def tasks(tasks, concurrency):
    N = min(len(tasks), concurrency)
    workers = []
    for i in range(N):
        worker = Worker()
        worker.start()
        workers.append(worker)
    for task in tasks:
        worker = workers.pop()
        worker.run(task)
    for worker in workers:
        worker.join()
    return { 'success': X, 'errors': Y }
```

## 七、Google Ads 侧的链接更新脚本

在 Google Ads MCC 中还需要一个脚本（和上广告的 bulk-manage 分开），定时从链接轮换表读取 G 列的参数，更新对应广告系列的 `finalUrlSuffix`。

### 核心逻辑

```
1. 获取链接轮换表数据
2. 遍历数据，找到需要更新的广告系列
3. 获取广告系列的 finalUrlSuffix
4. 调用 MccApp.select 方法
5. 调用 AdsApp.campaigns().withCondition 方法
6. 更新 finalUrlSuffix
7. if 广告系列 != G 广告系列:
    调用 finalUrlSuffix 方法
    调用 AdsApp.mutate 方法
```

**\*\*注意\*\***：只在参数真正变化时才更新，避免不必要的 API 调用。

## 八、完整数据流示例

假设你有一个已投放的广告，联盟名 `lh-12345-109748-ad`：

```
1. 获取广告 D 的 offer 数据
```



```
F ■ = 1■■■■■■■■■

2. link-resolver ■ 10 ■■■■■■
- ■ D ■■■■
- ■ C ■■■■■■■■■■ AU■
- ■ AU ■■■■■■■■ + ■■■■■■■■
- ■■■■■■■■■■
- ■■■■■■■■awc=12345_99999_xxxxxx
- ■■ G ■ + H ■■■■

3. Google Ads Script ■■■■■■
- ■ A ■ CID + C ■■■■ + G ■■■■
- ■■■■■■
- ■■■■■■
- ■■ finalUrlSuffix ■■■■■■
- ■■■■ → ■■

4. ■■■■■■■■ → ■■■■ → ■■
→ ■■ brand.com/?awc=12345_99999_xxxxxx
→ ■■■■■■■■■■■■■■ → ■■■■■■ ■
```

## 九、踩坑提醒

1. 住宅代理是核心瓶颈：质量差的代理会导致大量解析失败，选代理别省钱
2. Cookie 很关键：某些联盟的跳转链依赖 Set-Cookie，不带 Cookie 会导致死循环
3. Use

r-Agent                      要保持更新：半年更新一次                      UA  
池，太旧的版本号会被识别

4. 超时+重试缺一不可：代理请求 30 秒超时 + 3 次重试是基本配置
5. Google Ads Script 的 ES5 限制：同上广告系统，不能用现代 JS 语法
6. 日志要充分：每一跳都要打日志，方便排查解析失败的原因
7. 避免给联盟发太多请求：每 10 分钟解析一次已经够了，不要太频繁
8. Cron 时间编排：link-resolver 和 Google Ads Script 的执行时间要错开，确保 Ads Script 读到的是最新数据

## 十、关键依赖清单

| 依赖                   | 用途                   |
|----------------------|----------------------|
| Google Sheets API v4 | 读写链接轮转表              |
| Google Ads Script    | 更新广告的 finalUrlSuffix |
| 住宅代理服务               | 模拟不同国家的真实用户点击        |
| Node.js              | 运行 link-resolver 脚本  |

|      |                   |
|------|-------------------|
| Cron | 定时调度 (Linux 系统自带) |
|------|-------------------|

## 十一、推荐开发顺序

1. 先搭链接轮换表：设计好 A~H 列，手动填 2-3 行测试数据
2. 写链接解析核心函数：先不管并发和调度，把单条链接的解析跑通
3. 加住宅代理和指纹：确保解析出的参数是有效的（每次不同）
4. 加重试和超时：处理代理不稳定的情况
5. 接入 Sheets 读写：从 Sheets 读链接，结果写回 Sheets
6. 加并发池：5 个 worker 并发处理多条链接
7. 配 Cron 定时：每 10 分钟跑一次
8. 写 Google Ads 更新脚本：从 Sheets 读参数更新 finalUrlSuffix

## 十二、技术栈建议

- link-resolver：推荐 Node.js（`node-fetch` + `https-proxy-agent`）或 Python（`requests` + 代理配置）
- Goo

gle Ads 脚本：只能用 Google Ads Script（ES5 JavaScript）

- 定时调度：Cron（Linux）或 PM2 的 cron 模式